
Security Review Report
NM-0993 - Safe PASSKEY



NETHERMIND
SECURITY

(August 7, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Passkey (WebAuthn Signers)	4
5	Risk Rating Methodology	5
6	Issues	6
7	Documentation Evaluation	7
8	Test Suite Evaluation	8
8.1	Tests Output	8
9	About Nethermind	11

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Safe's](#) Passkey module. The review covered the module implementation and its use with the 1.3.0 version of the smart account.

The smart account is the base product, allowing users to create smart contracts that can be used as their accounts. It allows users to manage owners, signature thresholds, execute transactions from the Safe, and extend its functionalities through the use of modules.

The Passkey module contains contracts that can be deployed as owners of Safe accounts and implement WebAuth for signature verification. This extends the ways Safe accounts can authenticate their owners.

The audit comprises approximately 408 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report no points of attention.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

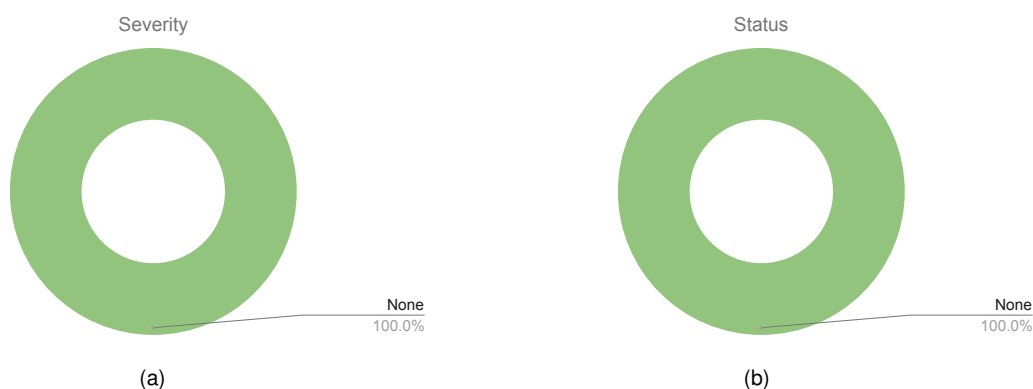


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (0), Best Practices (0). Distribution of status: Fixed (0), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	Aug 7, 2026
Final Report	Aug 17, 2026
Initial Commit (Passkey Module)	dfd3b05966e727dbb7a2fdeef52e4b230f63304e
Final Commit (Passkey Module)	dfd3b05966e727dbb7a2fdeef52e4b230f63304e
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	modules/passkey/contracts/SafeWebAuthnSignerSingleton.sol	16	21	131.2%	3	40
2	modules/passkey/contracts/SafeWebAuthnSignerFactory.sol	56	35	62.5%	9	100
3	modules/passkey/contracts/SafeWebAuthnSignerProxy.sol	33	42	127.3%	9	84
4	modules/passkey/contracts/verifiers/FCLP256Verifier.sol	26	11	42.3%	6	43
5	modules/passkey/contracts/libraries/WebAuthn.sol	121	205	169.4%	30	356
6	modules/passkey/contracts/libraries/P256.sol	74	83	112.2%	11	168
7	modules/passkey/contracts/libraries/ERC1271.sol	5	17	340.0%	2	24
8	modules/passkey/contracts/4337/SafeWebAuthnSharedSigner.sol	62	86	138.7%	19	167
9	modules/passkey/contracts/base/SignatureValidator.sol	15	24	160.0%	4	43
	Total	408	524	128.4%	93	1025

3 Summary of Issues

No findings were identified during the review.

4 System Overview

Safe is a multisignature smart contract account, it holds ETH, tokens, and arbitrary assets, and executes arbitrary transactions (CALL or DELEGATECALL) only when authorized. Authorization comes from one of two paths: an n-of-m threshold of owner signatures (execTransaction()), or a call from a whitelisted *module* contract, which bypasses signature checks entirely (execTransactionFromModule()). The account is extensible through modules, a transaction *guard*, and a *fallback handler*.

Deployment follows a proxy/singleton pattern: a single GnosisSafe implementation is deployed once per chain, and each user account is a minimal GnosisSafeProxy that stores the singleton address in storage slot 0 and delegatecalls all calls to it. Proxies are deployed by the GnosisSafeProxyFactory, optionally via CREATE2 for counterfactual addresses, and are initialized exactly once through setup(). GnosisSafeL2 is a subclass that only adds events so transactions can be indexed on chains.

4.1 Passkey (WebAuthn Signers)

The passkey contracts let users control Safes with WebAuthn credentials (passkeys) held by authenticators. Each passkey is represented on-chain by a dedicated ERC-1271 contract signer that is added as a regular Safe owner; contract-signature entries ($v == 0$) in checkSignatures() trigger full WebAuthn assertion verification against the credential's public key.

SafeWebAuthnSignerFactory deploys, via CREATE2, one minimal SafeWebAuthnSignerProxy per credential. The proxy holds the public key coordinates x, y and a packed P256.Verifiers configuration as immutables and forwards every call to a shared SafeWebAuthnSignerSingleton, appending the configuration to calldata. Using no storage keeps signers cheap and compliant with ERC-4337 validation-phase storage rules. The signature bytes are the ABI encoding of:

```

1 struct Signature {
2     bytes authenticatorData;
3     string clientDataFields;
4     uint256 r;
5     uint256 s;
6 }

```

- getSigner() / createSigner() - deterministic signer address computation and idempotent, permissionless deployment; the CREATE2 address commits to ($x, y, \text{verifiers}$).
- isValidSignatureForSigner() - verifies a signature for the specified address without deploying it.
- WebAuthn library - reconstructs the signed client data JSON with the ERC-1271 message hash base64url-encoded as the challenge, requires the user-verification (UV) authenticator flag, and computes the signing message authenticatorData || sha256(clientDataJSON) per the WebAuthn specification.
- P256 library - dispatches verification through a packed type Verifiers is uint176 value combining a 16-bit precompile address with a 160-bit fallback verifier address, so signers on chains without the precompile transparently upgrade once it goes live.
- FCLP256Verifier - the Solidity fallback verifier, wrapping the vendored FreshCryptoLib P-256 implementation
- SafeWebAuthnSharedSigner - a shared signer variant for ERC-4337 account creation: it is configured via delegatecall from the Safe and stores the credential in the Safe's own storage (allowed during 4337 validation), with the expectation of later migrating to a dedicated proxy signer.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

No findings were identified during the review.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Safe's documentation

The Safe team provided an overview of the main system components during the kick-off call with a detailed explanation of the intended functionalities. Additionally, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and an in-depth understanding of the project's technical aspects. The Safe team also provided previous audit reports to consult known design choices and later improvements to the protocol.

8 Test Suite Evaluation

8.1 Tests Output

```
> npx hardhat test

Safe4337Module
  SafeWebAuthnSigner
    executeUserOp - new account
      Used 772496 gas for >WebAuthn signer Safe operation<
      should execute user operation with a pre-deployed signer (470ms)
      Used 547956 gas for >WebAuthn signer Safe operation<
      should execute user operation from a 1/2 Safe with a passkey owner
    executeUserOp - existing account
      Used 504755 gas for >WebAuthn signer Safe operation<
      should execute user operation
  SafeWebAuthnSharedSigner
    executeUserOp - new account
      Used 841854 gas for >WebAuthn signer Safe deployment<
      should execute user operation
    executeUserOp - existing account
      Used 510487 gas for >WebAuthn signer Safe operation<
      should execute user operation

SafeWebAuthnSharedSigner
  getConfiguration
    Should return a configuration for an account (51ms)
    Should return empty configuration for a non-configured account
    Should return empty configuration if getting storage for account reverts
    Should return empty configuration for non-standard ABI encoded storage result
    Should store the signer configuration in a mapping with the shared signer address as a key
  configure
    Should configure an account
    Should emit an event
    Should revert if not DELEGATECALL-ed
  isValidSignature
    Should return a magic value for an account
    Should not verify signature if invalid
    Should not verify signature if account not configured

Safe WebAuthn Shared Signer [@4337]
  - should execute a user op with the WebAuthn shared signer
  - should execute a user op and deploy a new WebAuthn signer
  - should execute a user op with multiple passkey owners

Safe WebAuthn Signer [@4337]
  - should execute a user op with a deployed WebAuthn signer

Gas Benchmarking [@bench]
  SafeWebAuthnSignerProxy
    deployment: 89900
    Benchmark signer deployment cost
    verification (Daimo): 344941
    Benchmark signer verification cost with Daimo verifier
    verification (Dummy): 12581
    Benchmark signer verification cost with Dummy verifier
    - Benchmark signer verification cost with Precompile verifier

SafeWebAuthnSignerFactory
  getSigner
    Should return the address that a signer will be created on
    Should return different signer for different inputs
  createSigner
    Should create a signer and return its deterministic address
    Should be idempotent
    Should emit event only once
  isValidSignatureForSigner
    Should return true when the verifier returns true
    Should return true when the verifier without precompile returns true
    Should return true when the precompile returns true
    Should return false when the verifier does not return true
```

Should **return false** on non-matching authenticator flags

SafeWebAuthnSignerProxy

fallback

Should forward call to singleton with additional information

constructor

Should **set** immutables

isValidSignature

Should **return true** when the verifier returns **true**

Should **return true** when the precompile returns **true**

Should **return false** when the verifier does not **return true**

Should **return false** on non-matching authenticator flags

P256

isValidSignature

Should **return true** on valid signature

Should **return false** on invalid signature

Should check **for** signature signature malleability

Should **return false for** misbehaving verifiers

isValidSignatureWithVerifiers

Should handle all possible verifier configurations

Should fallback to Solidity verifier when precompile fails

Should **return false** on invalid signature

Should check **for** signature signature malleability

Should **return false for** misbehaving verifiers

WebAuthn Library

castSignature

Should correctly cast an ABI encoded WebAuthn signature

Should correctly cast a packed encoded WebAuthn signature

Should correctly cast a partially packed encoded WebAuthn signature

Should detect encoded WebAuthn signatures with too much padding

encodeClientDataJson

Should correctly base-64 encode the 32 byte challenge

encodeSigningMessage

Should correctly compute a signing message

Should revert **if** SHA-256 precompile reverts

verifySignature

Should **return false** when the verifier returns **false**

Should **return false** on non-matching authenticator flags

Should **return true** when the verifier returns **true**

Should **return false** when the signature data has too much padding

Create a Safe with Passkey signer as owner: [Userstory]

should execute a userOp with WebAuthn signer as owner (60ms)

Deterministic passkey signer address [Userstory]

should execute a userOp with replaced WebAuthn signer as Safe owner (45ms)

Execute Safe Transaction [Userstory]

should create a Safe and execute a transaction (52ms)

should create a Safe and execute a transaction with the shared WebAuthn signer

Execute userOp from Passkey signer [Userstory]

should execute a userOp with WebAuthn signer as owner (54ms)

Execute userOps with Paymaster: [Userstory]

New Safe

should execute a userOp and deploy a Safe using Paymaster (59ms)

Existing Safe

should execute a userOp with an existing Safe using Paymaster (58ms)

Offchain Passkey Signature Verification [Userstory]

should be possible to verify offchain passkey signature (50ms)

Passkey Credential Creation **for** Safe Ownership [Userstory]

should be possible to execute a transaction signed by passkey (51ms)

Rotate passkey owner [Userstory]

should execute a userOp with replaced WebAuthn signer as Safe owner (49ms)

Safe Address **for** Passkey [Userstory]

should compute the Safe address owned by a WebAuthn proxy signer (47ms)

should compute the Safe address owned by a WebAuthn shared signer

should search **for** Safes owned by a WebAuthn shared signer

DaimoP256Verifier

- Should **return** 1 on valid signature
- Should ignore signature malleability
- Should **return** empty on unverified signature
- Should **return** empty on invalid signature parameters
- Should **return** empty on invalid public key
- Should **return** empty on invalid input

WebAuthn Shim

- navigator.credentials.create()
 - Should create and verify a new credential
- navigator.credentials.get()
 - Should authorise and verify an existing credential

76 passing (1s)

5 pending

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.